Modules **Django Fast Track** 0 of 20 done Show all answers

FROM BASIC DJANGO TO ENTERPRISE LEVEL

Learn Django fast: short notes, clear flows, real practice.

20 short modules. Each one gives you **What**, **Why**, **When**, **How**, a diagram, small code, common mistakes and interview questions. Do them in order. Tick each module when done; your progress is saved in this browser.

`Django 6.1 (latest)` `Django 5.2 LTS → Apr 2028` `DRF 3.17` `Celery 5.6` `Python 3.12+ for Django 6`

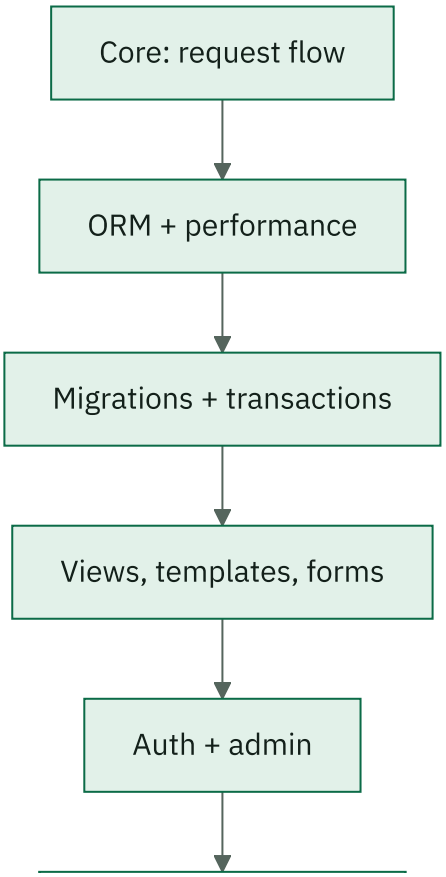
00 Start here

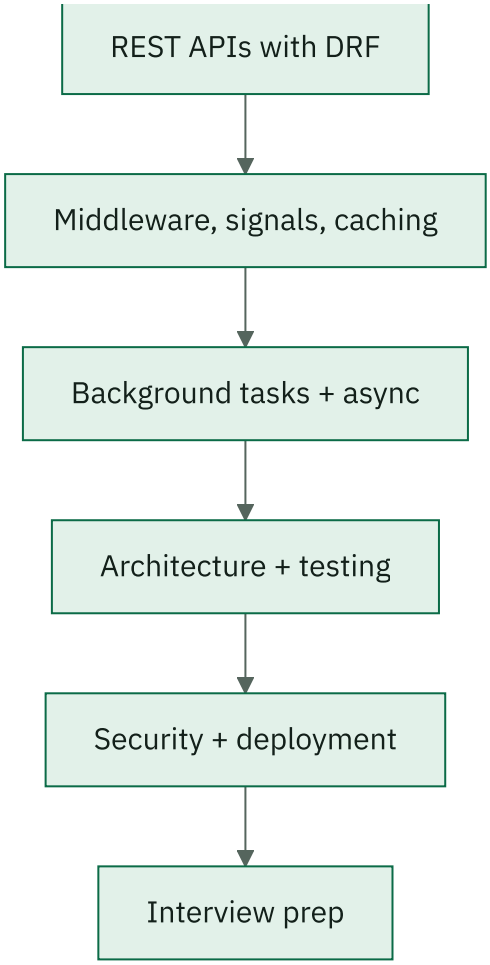
The route, the versions, and how to use this course.

~10 min ☐ Mark as done

The learning route

YOUR PATH





Which Django version should you learn?

LEARN

Django 6.1 (latest) or 5.2 LTS. The core ideas are the same.

WHY

Companies use LTS (long-term support) versions. Interviewers like it when you know what is new.

YOUR SETUP

Your Folder 12 uses Django 2.1 on Python 3.8. That is very old. Start a fresh project for this course.

RULE

Python 3.11 → use Django 5.2 LTS. Python 3.12+ → use Django 6.1.

Copy

```
# New project for this course (you have Python 3.11.9 in pyenv)
mkdir vidly-pro && cd vidly-pro
~/.pyenv/versions/3.11.9/bin/python -m venv .venv
source .venv/bin/activate
python -m pip install "django~=5.2" djangorestframework
django-admin startproject config .
python manage.py runserver
```

HOW EACH MODULE WORKS

- **What / Why / When / How**: the idea in 4 boxes. Read these first.
- **Diagram**: the flow. If you understand the diagram, you understand the topic.
- **Code**: only the code you need. Type it, don't just read it.
- **Watch out**: common mistakes that cost time in real projects.
- **Interview**: try to answer out loud first, then open the answer.
- **Practice**: each task grows one project, **Vidly Pro**, a movie rental app and API.

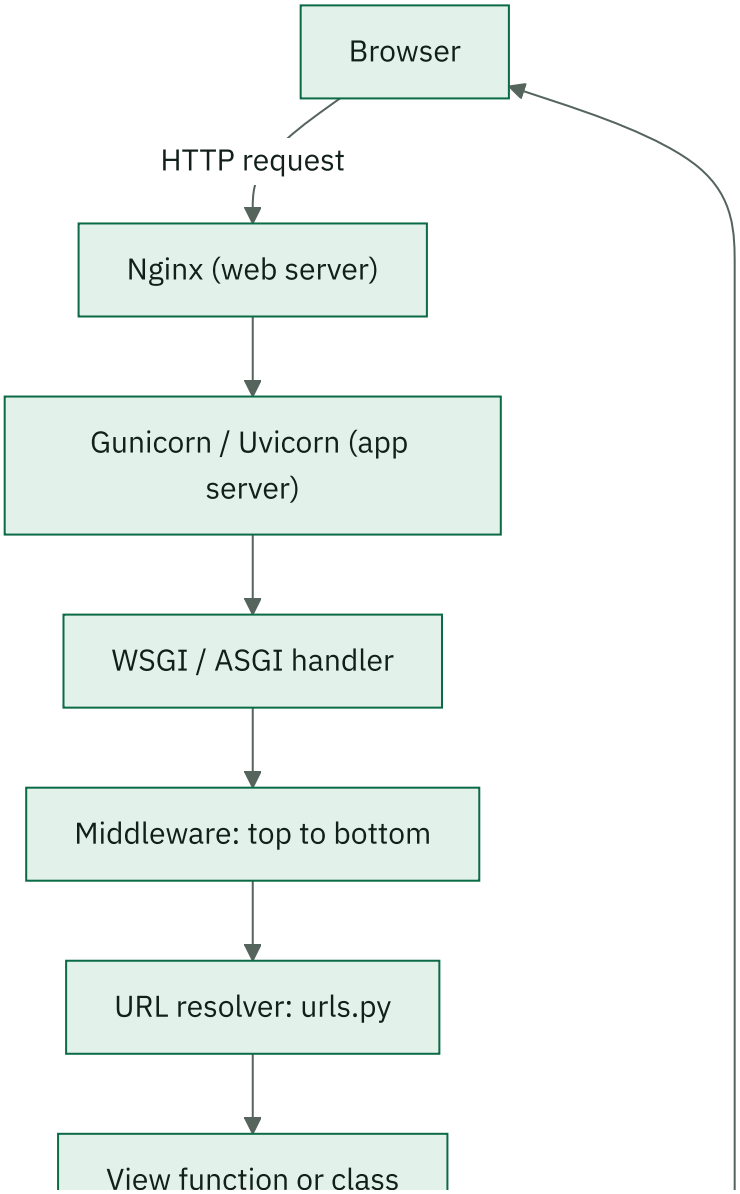
01 How Django works

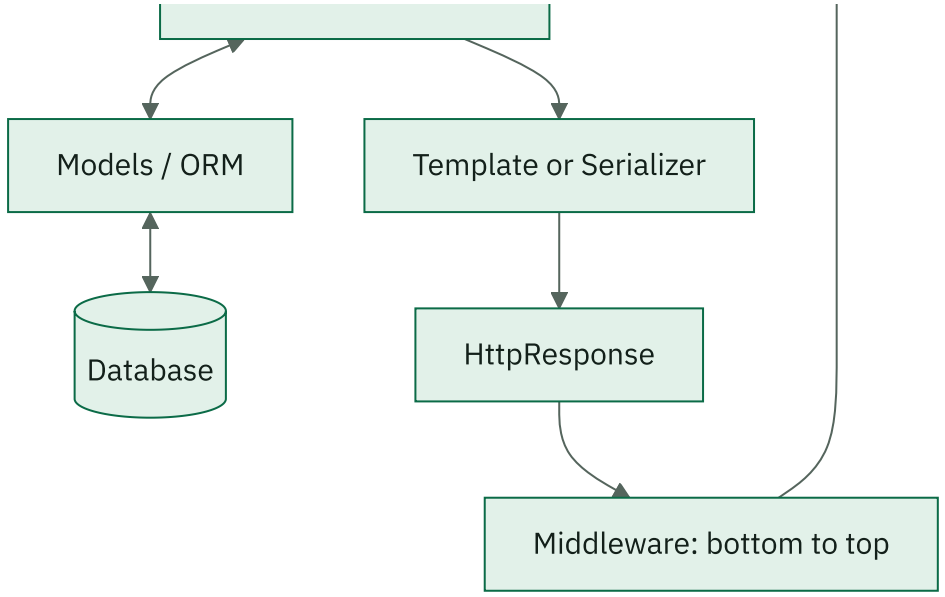
One request, start to finish. Everything else plugs into this flow.

~20 min ☐ Mark as done

The request/response cycle

WHAT HAPPENS WHEN SOMEONE OPENS /MOVIES/1/





- **Middleware** wraps every request, like layers of an onion (sessions, auth, CSRF).
- **URL resolver** matches the path to a view.
- **View** holds the request logic. It reads or writes data through the ORM.
- **Template** (HTML) or **serializer** (JSON) builds the output.

MVT: Model, View, Template

WHAT

Django's version of MVC. Model = data, View = logic, Template = HTML.

WHY

Each part has one job, so code stays easy to change.

KEY POINT

Django's "View" is like the MVC "Controller". Django itself (URLs + framework) does the routing.

HOW

`models.py` → `views.py` → `templates/`, connected by `urls.py`.

Project vs app

PROJECT

The whole website: settings, root URLs, WSGI/ASGI entry points.

APP

One feature area (movies, rentals, users). It has its own models, views, URLs and tests.

WHEN TO MAKE A NEW APP

When a feature has its own data and can be explained in one sentence.

HOW

`python manage.py startapp rentals`, then add it to `INSTALLED_APPS`.

Important files

FILE	JOB
`manage.py`	Command-line tool: runserver, migrate, shell, test.
`settings.py`	All configuration: apps, database, middleware, templates, static.
`urls.py`	Maps URL paths to views.
`wsgi.py` / `asgi.py`	Entry points for production servers (sync / async).
`apps.py`	App config. `ready()` runs once at startup (good place to connect signals).

WATCH OUT

- Never use `runserver` in production. It is for development only.
- App order in `INSTALLED_APPS` matters for templates and static files: the first match wins.

INTERVIEW

Explain the Django request lifecycle. Show answer

What is MVT and how is it different from MVC? Show answer

Project vs app? Show answer

PRACTICE: VIDLY PRO

1. Create the new project (see Module 00) and an app called `movies`.
2. Write one view that returns `HttpResponse("hello")` and connect it in `urls.py`.
3. Add a `print()` in the view, and trace the request in the terminal.

02 Models and the ORM

Python classes become tables. QuerySets become SQL.

~40 min ☐ Mark as done

Models, fields and relationships

WHAT WHY

A model is a Python class. One class = one table. One field = one column.

You write Python instead of SQL. Django handles escaping, joins and different databases.

WHEN

For any data you store. Use raw SQL only for rare, very complex queries.

HOW

Subclass `models.Model`, add fields, then run makemigrations + migrate.

```
from django.db import models

class Genre(models.Model):
    name = models.CharField(max_length=100, unique=True)

class Movie(models.Model):
    title = models.CharField(max_length=255)
    genre = models.ForeignKey(Genre, on_delete=models.PROTECT, related_name="m
    tags = models.ManyToManyField("Tag", blank=True)
    daily_rate = models.DecimalField(max_digits=6, decimal_places=2)  # money!
    number_in_stock = models.PositiveIntegerField(default=0)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["-created_at"]
        indexes = [models.Index(fields=["title"])]
        constraints = [
            models.CheckConstraint(condition=models.Q(daily_rate__gte=0), name
        ]

    def __str__(self):
        return self.title
```

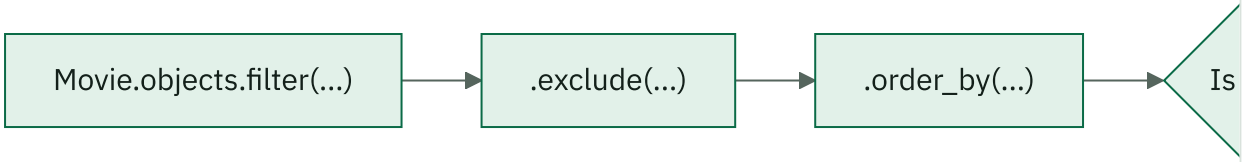
RELATIONSHIP	FIELD	EXAMPLE
Many-to-one	`ForeignKey`	Many movies → one genre
One-to-one	`OneToOneField`	User → Profile
Many-to-many	`ManyToManyField`	Movies ↔ Tags. Use `through=` when the link needs extra fields.

ON_DELETE	WHAT HAPPENS WHEN THE PARENT IS DELETED
`CASCADE`	Delete the children too.
`PROTECT`	Block the delete (raises an error). Safe default for important data.

`SET_NULL`	Set the field to NULL (needs `null=True`).
`RESTRICT`	Like PROTECT, but allows the delete if the parent is removed by another CASCADE.
`DB_CASCADE` (6.1)	The database does the cascade. Faster, but no Django signals run.

QuerySets are lazy

CHAINING BUILDS A QUERY; IT RUNS ONLY WHEN NEEDED



Copy

```
from django.db.models import Q, F, Count, Avg

Movie.objects.filter(daily_rate__gte=3, title__icontains="war")     # AND
Movie.objects.filter(Q(genre__name="Comedy") | Q(daily_rate__lt=2))  # OR with
Movie.objects.filter(genre__name="Drama")                           # follow
Movie.objects.get(pk=1)              # exactly one, else DoesNotExist / Multi
Movie.objects.filter(pk=1).first()  # one or None, never raises

# F() = use a column value inside the database (no race condition)
Movie.objects.filter(pk=1).update(number_in_stock=F("number_in_stock") - 1)

# annotate = add a value PER ROW; aggregate = ONE value for the whole set
Genre.objects.annotate(movie_count=Count("movies"))
Movie.objects.aggregate(avg_rate=Avg("daily_rate"))   # {'avg_rate': Decimal(
```

Custom managers and QuerySets

WHAT

Your own reusable query methods, such as `Movie.objects.available()`.

WHY

One place for common filters. They stay chainable.

WHEN

When the same filter appears in 2+ places.

HOW

Subclass `QuerySet`, then use `.as_manager()`.

```python
class MovieQuerySet(models.QuerySet):
    def available(self):
        return self.filter(number_in_stock__gt=0)

    def cheap(self):
        return self.filter(daily_rate__lt=2)

class Movie(models.Model):
    ...
    objects = MovieQuerySet.as_manager()

Movie.objects.available().cheap()   # chainable
```

Copy

Model inheritance: 3 types

TYPE	TABLES	USE IT WHEN
Abstract base (`abstract = True`)	No table for the parent. Fields are copied into each child.	Shared fields such as `created_at` . **Most common.**
Multi-table	Parent and child tables, joined	Rarely. Every read needs a JOIN.
Proxy (`proxy = True`)	Same table	Same data with different behavior, ordering or admin.

WATCH OUT

- `null=True` is about the database (allows NULL). `blank=True` is about validation (allows empty in forms). For text fields, use `blank=True` only, and store "" instead of NULL.
- Use `DecimalField` for money. `FloatField` gives rounding errors.
- `queryset.update()` and `bulk_create()` skip `save()` and signals.

INTERVIEW

When is a QuerySet evaluated? Show answer

Difference between F() and Q()? Show answer

annotate() vs aggregate()? Show answer

get() vs filter()? Show answer

PRACTICE: VIDLY PRO

1. Create the `Genre`, `Movie`, `Customer` and `Rental` models (Rental: customer FK, movie FK, rented_at, returned_at).
2. Add `MovieQuerySet.available()`.
3. In `python manage.py shell`, count movies per genre with `annotate`.

03 Query performance

The number one interview topic. Slow Django apps are almost always slow queries.

~35 min ☐ Mark as done

The N+1 problem

WHAT

1 query for the list, then 1 extra query for each row's related object.

WHY IT HURTS

100 movies = 101 queries. The page gets slower as the data grows.

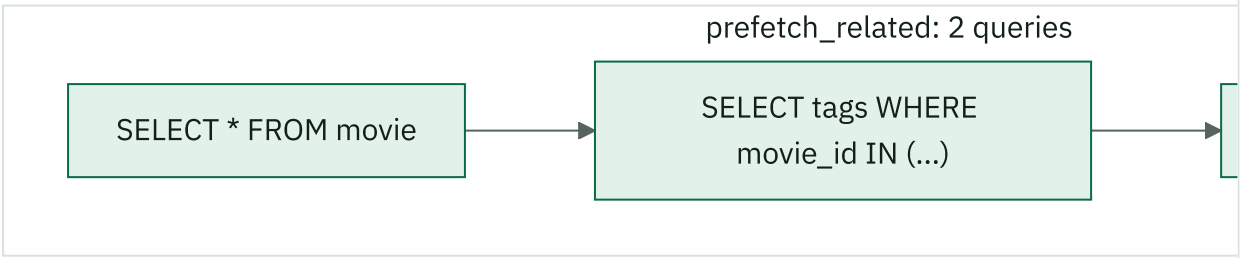
WHEN IT HAPPENS

When a loop (in a template or serializer) accesses `movie.genre` or `movie.tags.all()`.

FIX

`select_related` for FK/one-to-one; `prefetch_related` for many-to-many and reverse FK.

SAME PAGE, THREE WAYS



Copy

```
from django.db.models import Prefetch

# FK / OneToOne -> SQL JOIN
movies = Movie.objects.select_related("genre")

# M2M / reverse FK -> second query + Python join
movies = Movie.objects.prefetch_related("tags")
```

```
# Prefetch with a custom queryset (only active rentals)
customers = Customer.objects.prefetch_related(
    Prefetch("rentals", queryset=Rental.objects.filter(returned_at__isnull=Tru
)
```

Load less data

TOOL	USE IT TO
`only("id", "title")` / `defer("plot")`	Load fewer columns.
`values()` / `values_list(..., flat=True)`	Get dicts or tuples instead of model objects. Faster for reports.
`exists()`	Check "is there any?" without loading rows.
`count()`	Count in SQL. Don't use `len(qs)` unless you need the rows anyway.
`bulk_create` / `bulk_update`	Insert or update many rows in one query.
`iterator(chunk_size=2000)`	Loop over millions of rows without filling memory.
`qs.explain()`	See the database's query plan.

Indexes

WHAT
A sorted lookup structure, like a book index. It makes reads fast.

WHY
Without an index, the database scans every row.

WHEN
For columns used often in `filter`, `order_by` or joins. FKs get an index automatically.

COST
Writes get slower and the database uses more disk. Don't index everything.

Copy

```
class Meta:
    indexes = [
        models.Index(fields=["genre", "-created_at"]),           # composite:
        models.Index(fields=["title"], condition=models.Q(number_in_stock__gt=
                     name="available_title_idx"),                 # partial i
    ]
```

NEW IN DJANGO 6.1: FETCH MODES

`Movie.objects.fetch_mode(models.FETCH_PEERS)` turns a hidden N+1 into 2 queries automatically. `FETCH_RAISE` raises an error on any unplanned query, which is great in development and tests to catch N+1 early.

How to find slow queries

- **Django Debug Toolbar**: shows every SQL query per page in development.
- **django-silk**: profiles requests and queries.
- **Tests**: `assertNumQueries(2)` locks the query count so N+1 can't come back.
- **Production**: APM tools (Sentry, New Relic, Datadog) and the database's slow query log.

WATCH OUT

- `select_related` on a many-to-many does nothing useful. Use `prefetch_related`.
- Calling `.filter()` on a prefetched relation runs a new query and ignores the prefetch. Use `Prefetch(..., queryset=...)` instead.
- Measure first. Don't guess.

INTERVIEW

select_related vs prefetch_related? Show answer

How do you find and fix a slow Django page? Show answer

When would you use raw SQL? Show answer

PRACTICE: VIDLY PRO

1. Install `django-debug-toolbar`. Load a movie list page that shows the genre and tags.
2. Count the queries, fix them with select/prefetch, and count again.
3. Write a test with `assertNumQueries` for that view.

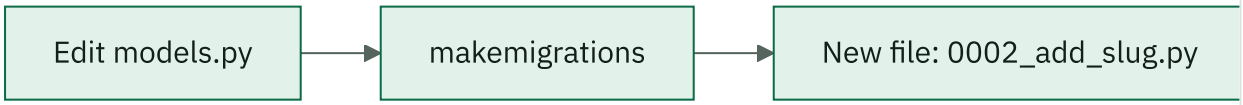
04 Migrations

Version control for your database schema.

~25 min ☐ Mark as done

How migrations work

FROM MODEL CHANGE TO DATABASE CHANGE



COMMAND	DOES
`makemigrations`	Compares the models with the last migration and writes a new file.
`migrate`	Applies the files that haven't run yet.
`showmigrations`	Lists applied [X] and pending [ ] migrations.
`sqlmigrate app 0002`	Shows the SQL a migration will run. Check it before big changes.
`migrate app 0001`	Rolls back to migration 0001.
`squashmigrations`	Combines many old files into one.

Data migrations

WHAT

A migration that changes data, not the schema.

WHEN

To fill a new column, fix bad values or split a field.

HOW

`makemigrations movies --empty -n fill_slugs`, then add `RunPython`.

RULE

Use `apps.get_model()`, never a direct import. The model must match that point in history.

Copy

```
from django.db import migrations
from django.utils.text import slugify

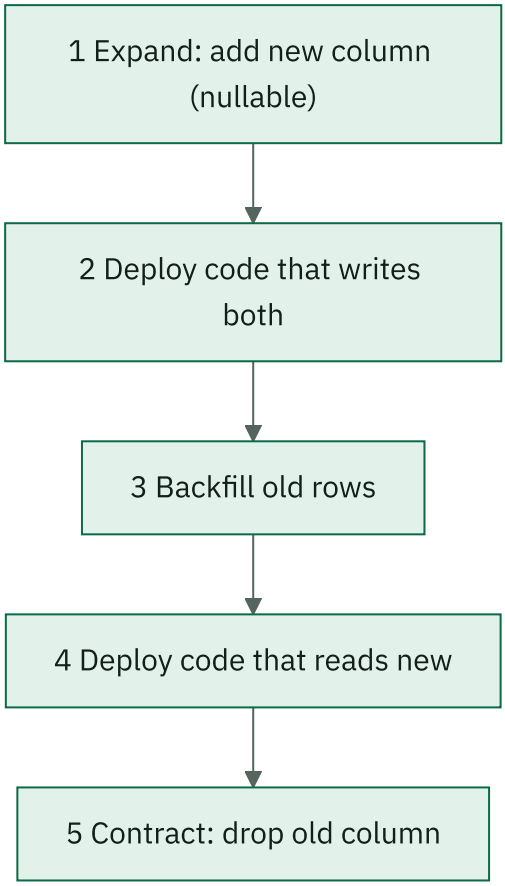
def fill_slugs(apps, schema_editor):
    Movie = apps.get_model("movies", "Movie")
    for movie in Movie.objects.filter(slug=""):
        movie.slug = slugify(movie.title)
        movie.save(update_fields=["slug"])

class Migration(migrations.Migration):
    dependencies = [("movies", "0002_movie_slug")]
    operations = [migrations.RunPython(fill_slugs, migrations.RunPython.noop)]
```

operations = [migrations.RunPython(fill_slugs, migrations.RunPython.noop)]

Zero-downtime changes (expand and contract)

RENAMING OR REPLACING A COLUMN SAFELY IN PRODUCTION



Why: during a deploy, old and new code run at the same time. Each step must work with both.

WATCH OUT

- Never edit a migration that already ran in production. Write a new one.
- Adding a NOT NULL column without a default, or adding an index to a huge table, can lock the table. On PostgreSQL, use `AddIndexConcurrently`.
- Two developers' migrations with the same number cause a conflict. Fix it with `makemigrations --merge`.
- Always commit migration files to git.

INTERVIEW

How do you run migrations safely in production? Show answer

Schema migration vs data migration? Show answer

PRACTICE: VIDLY PRO

1. Add an empty `slug` field to Movie and migrate.
2. Write a data migration that fills the slugs, then make the field `unique=True` in a third migration.
3. Read the SQL with `sqlmigrate` .

05 Transactions and data safety

All or nothing. No double bookings. No lost updates.

~25 min ☐ Mark as done

transaction.atomic

WHAT

A block of database work that fully succeeds or fully rolls back.

WHY

If step 2 fails, step 1 must not stay saved (for example, payment saved but rental not).

WHEN

Any action that writes 2 or more rows that belong together.

HOW

`@transaction.atomic` or `with transaction.atomic():`

Race conditions and locking

Two users rent the last copy at the same moment. Both read "stock = 1". Both rent it. Now stock is −1. Two fixes:

Copy

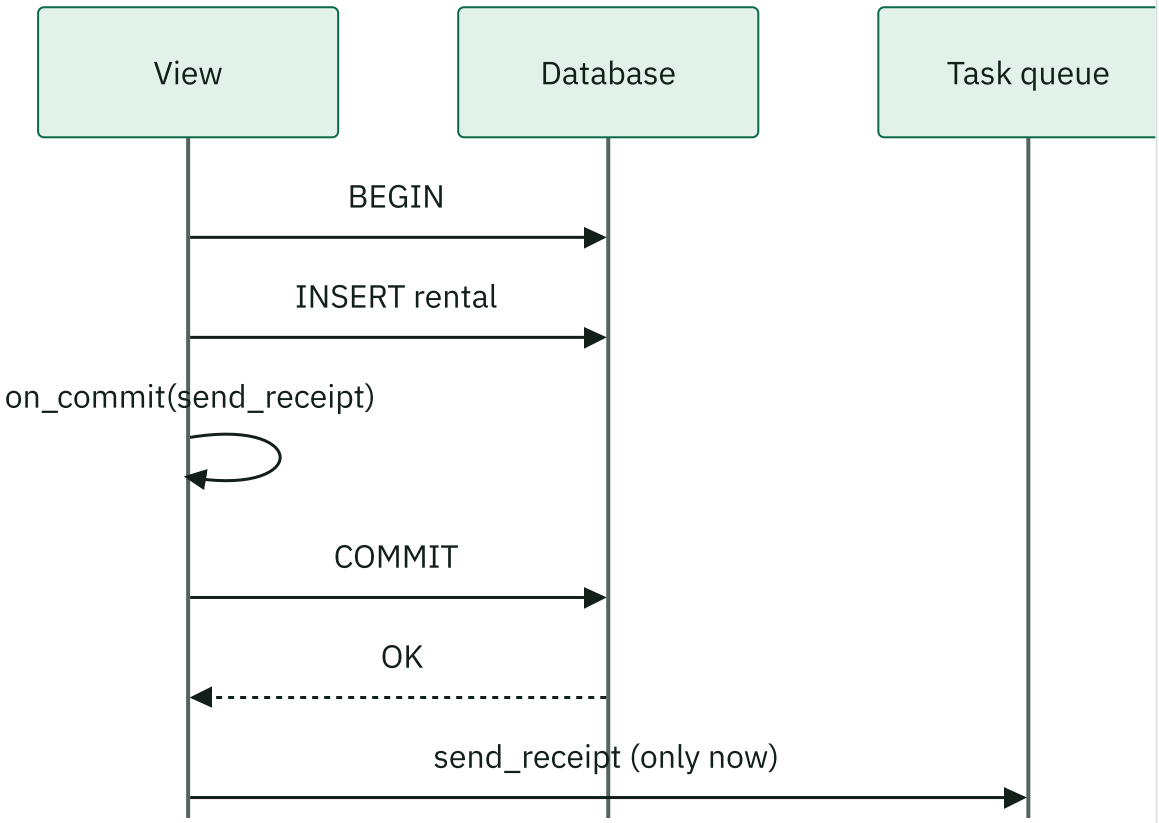
```
from django.db import transaction
from django.db.models import F

# Fix A: lock the row until the transaction ends
@transaction.atomic
def rent_movie(customer, movie_id):
    movie = Movie.objects.select_for_update().get(pk=movie_id)    # other reque
    if movie.number_in_stock < 1:
        raise OutOfStock()
    movie.number_in_stock -= 1
    movie.save(update_fields=["number_in_stock"])
    rental = Rental.objects.create(customer=customer, movie=movie)
    transaction.on_commit(lambda: send_receipt.enqueue(rental_id=rental.pk))
    return rental
```

```
# Fix B: one atomic UPDATE with a condition (no lock needed)
updated = Movie.objects.filter(pk=movie_id, number_in_stock__gt=0).update(
    number_in_stock=F("number_in_stock") - 1
)
if updated == 0:
    raise OutOfStock()
```

on_commit: run side effects only after save

WHY EMAILS AND TASKS WAIT FOR COMMIT



If you queue the task before the commit, the worker may not find the row yet, or the transaction may roll back after the email has already been sent.

WATCH OUT

- `ATOMIC_REQUESTS = True` wraps every request in a transaction. It's simple, but it holds locks longer. Many teams prefer explicit `atomic` blocks.
- Python validation is not enough. Add `UniqueConstraint` or `CheckConstraint` so the database is the final guard.
- `get_or_create` can still race. Back it with a unique constraint.

INTERVIEW

How do you prevent two users booking the same item? Show answer

Why use transaction.on_commit? Show answer

PRACTICE: VIDLY PRO

1. Write `rent_movie()` with `select_for_update`.
2. Add a `CheckConstraint` so stock can never go below 0.
3. Add a `UniqueConstraint`: a customer can't have two open rentals of the same movie.

06 Views and URLs

Function views for clarity, class-based views for reuse.

~30 min ☐ Mark as done

FBV vs CBV

	FUNCTION-BASED VIEW (FBV)	CLASS-BASED VIEW (CBV)
Looks like	A plain function	A class with methods (get, post)
Good for	Custom, one-off logic. Easy to read.	Standard CRUD pages. Reuse through mixins.
Downside	Repeated code for CRUD	"Hidden" flow, so you must know the methods to override.

Copy

```
# FBV
def movie_detail(request, pk):
    movie = get_object_or_404(Movie, pk=pk)
    return render(request, "movies/detail.html", {"movie": movie})

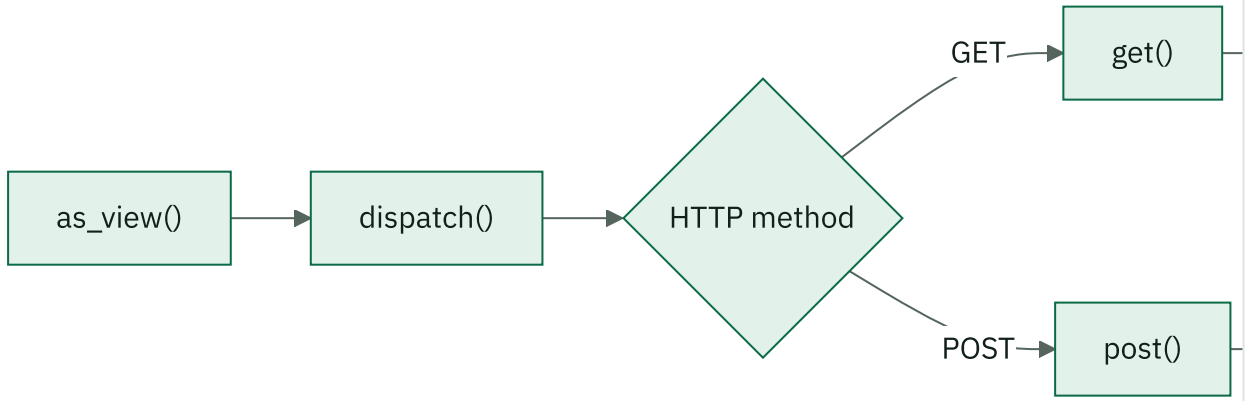
# CBV: same page, plus a list page, in a few lines
from django.views.generic import ListView, DetailView
from django.contrib.auth.mixins import LoginRequiredMixin

class MovieListView(LoginRequiredMixin, ListView):
    queryset = Movie.objects.select_related("genre")
    paginate_by = 20

class MovieDetailView(DetailView):
```

```
model = Movie    # template: movies/movie_detail.html
```

HOW A CBV HANDLES A REQUEST



URLs

Copy

```
# movies/urls.py
app_name = "movies"                          # namespace
urlpatterns = [
    path("", views.MovieListView.as_view(), name="list"),
    path("<int:pk>/", views.MovieDetailView.as_view(), name="detail"),
    path("<slug:slug>/", views.movie_by_slug, name="by-slug"),
]

# config/urls.py
urlpatterns = [path("movies/", include("movies.urls"))]

# Never hard-code URLs:
reverse("movies:detail", args=[5])       # in Python
{% url 'movies:detail' movie.pk %}       # in templates
```

- Converters: **`int`**, **`str`**, **`slug`**, **`uuid`**, **`path`**.
- **`reverse_lazy`** is for class attributes (such as **`success_url`**), because they load before URLs are ready.

WATCH OUT

- Mixin order matters: put **`LoginRequiredMixin`** first (on the left).
- Use **`get_object_or_404`**, not **`.get()`**, in views. Otherwise a missing row gives a 500 error instead of a 404.

INTERVIEW

When FBV, when CBV? Show answer

What does as_view() do? Show answer

PRACTICE: VIDLY PRO

1. Build movie list and detail pages with `ListView` and `DetailView`, with pagination.
2. Add a genre filter through the query string: `?genre=comedy` in `get_queryset()`.

07 Templates and forms

Safe HTML output and safe user input.

~30 min ☐ Mark as done

Templates: the essentials

- **Inheritance**: `{% extends "base.html" %}` + `{% block content %}`.
- **Include**: `{% include "partials/card.html" with movie=m %}`.
- **Auto-escaping**: `{{ value }}` is HTML-escaped, which blocks XSS. `|safe` turns that off, so only use it on content you trust.
- **Context processors**: add variables to every template (for example `request` and `user`).
- **Custom tags and filters**: put them in `app/templatetags/`.

NEW IN DJANGO 6.0: TEMPLATE PARTIALS

Define a named fragment once with `{% partialdef movie-row %}…{% endpartialdef %}`, reuse it with `{% partial movie-row %}`, or render only that piece with `render(request, "movies.html#movie-row", ctx)`. This is great with HTMX.

Forms

WHAT

Classes that validate input and render HTML fields.

WHY

Never trust user input. Forms clean the data, check it and report errors.

WHEN

`ModelForm` to create or edit a model.

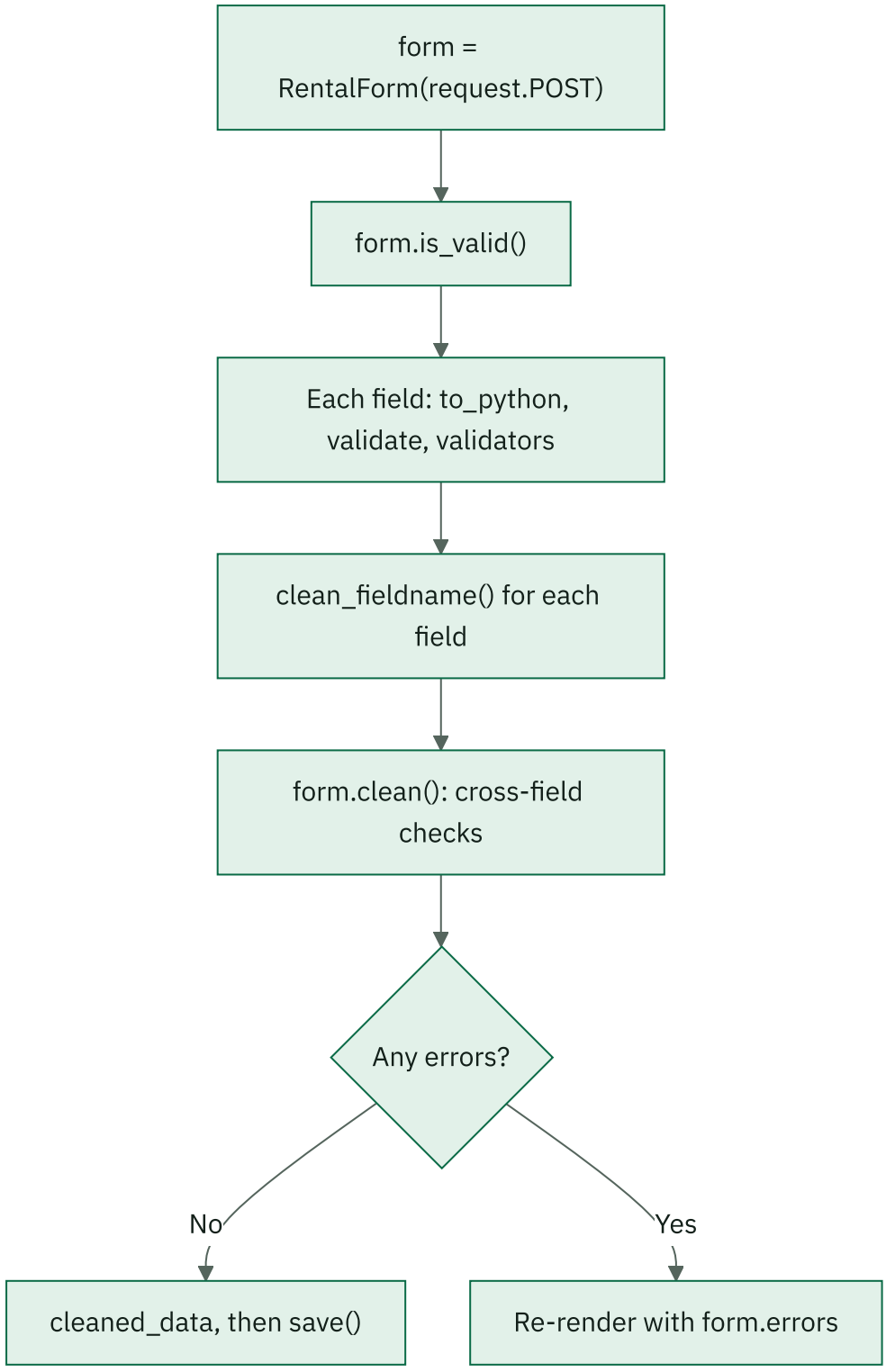
HOW

`is_valid()`, then use `cleaned_data` or

`Form` for anything else (search, contact).

`save()`.

VALIDATION ORDER



Copy

```
class RentalForm(forms.ModelForm):
    class Meta:
        model = Rental
        fields = ["movie", "rented_at", "due_at"]

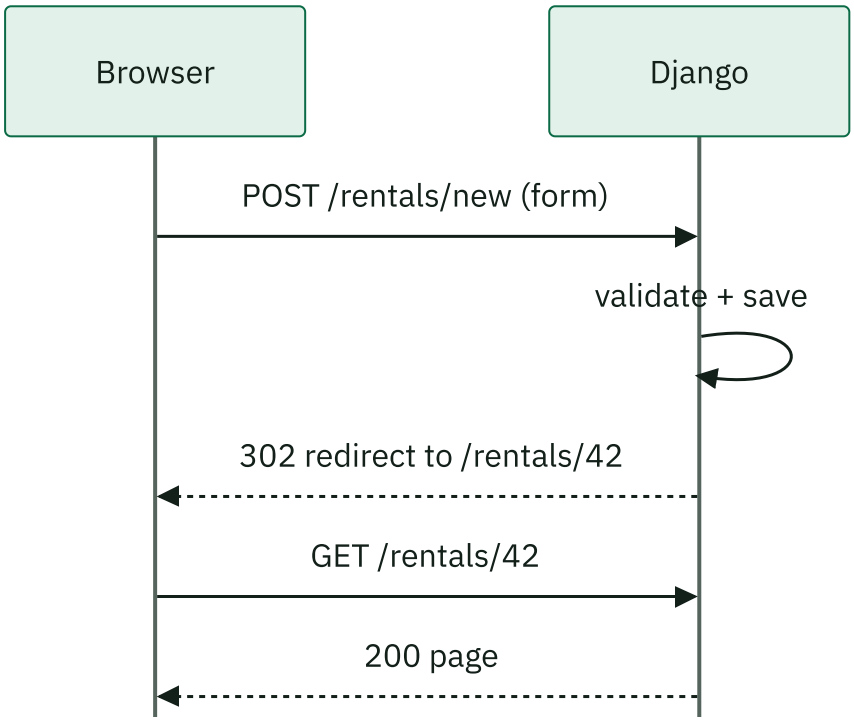
    def clean_movie(self):                    # one field
        movie = self.cleaned_data["movie"]
```

```
        if movie.number_in_stock < 1:
            raise forms.ValidationError("Out of stock.")
        return movie

    def clean(self):                                # several fields
        data = super().clean()
        if data.get("due_at") and data["due_at"] <= data["rented_at"]:
            self.add_error("due_at", "Due date must be after rent date.")
        return data
```

POST, Redirect, GET (PRG)

STOPS DOUBLE SUBMITS ON REFRESH



WATCH OUT

- Every POST form needs `{% csrf_token %}`.
- In ModelForm, always list `fields` explicitly. Using `"__all__"` can let users set fields they shouldn't (mass assignment).

INTERVIEW

How does CSRF protection work? Show answer

clean_<field>() vs clean()? Show answer

PRACTICE: VIDLY PRO

1. Build a rental create page with `CreateView + RentalForm`, including the due-date rule.
2. Show a success message with the messages framework after the redirect.

08 Users, auth and permissions

Who are you (authentication) and what can you do (authorization).

~35 min ☐ Mark as done

Custom user model: do it on day one

WHAT
Your own `User` class instead of Django's default.

WHY
Changing the user model later is very painful, because every FK points to it.

WHEN
Before the first `migrate` of every new project, even if it's empty.

HOW
Subclass `AbstractUser` and set `AUTH_USER_MODEL`.

Copy

```
# users/models.py
from django.contrib.auth.models import AbstractUser

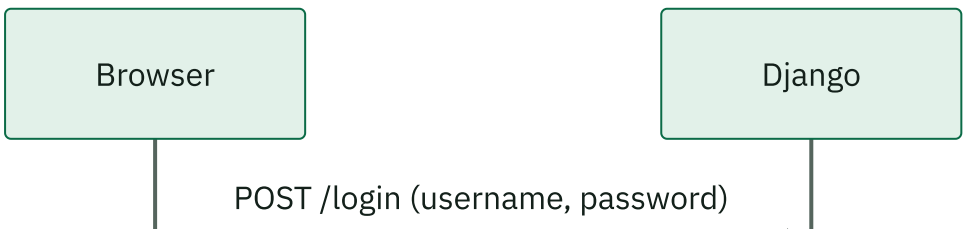
class User(AbstractUser):
    phone = models.CharField(max_length=20, blank=True)

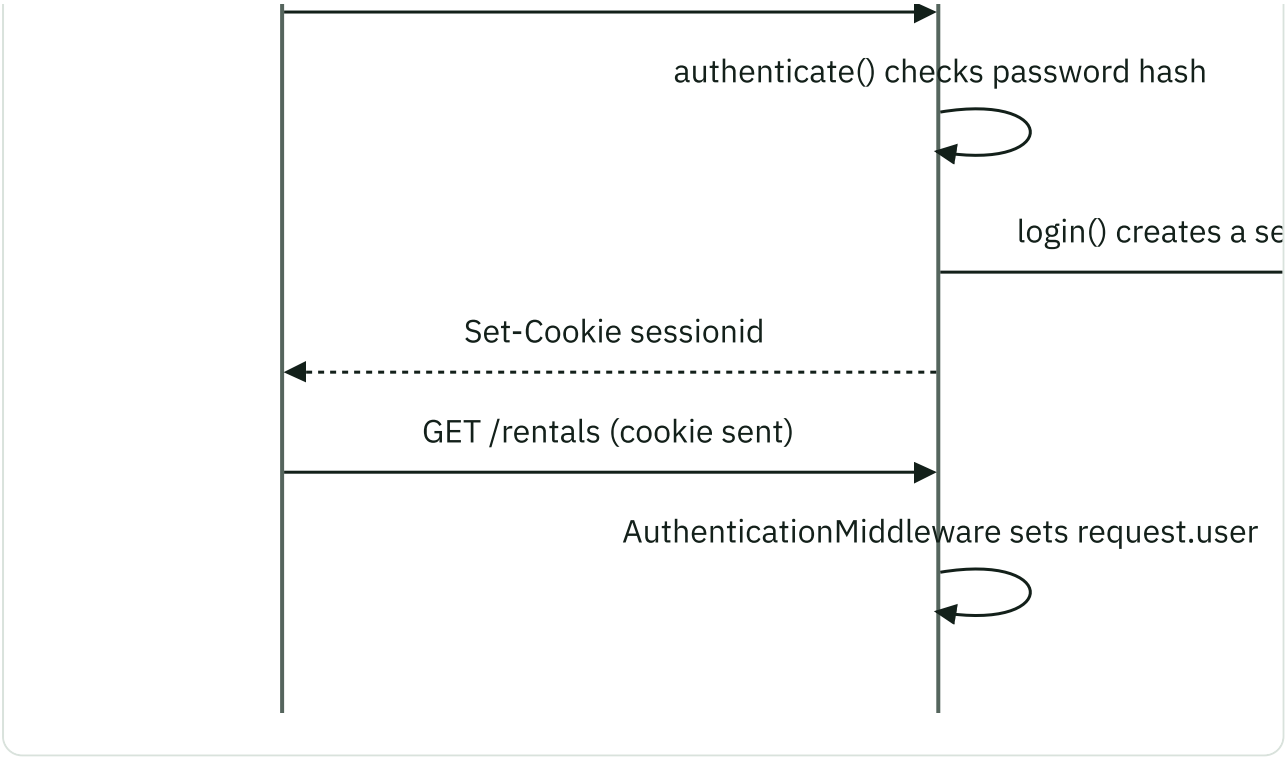
# settings.py
AUTH_USER_MODEL = "users.User"

# everywhere else: never import User directly
from django.contrib.auth import get_user_model
User = get_user_model()
customer = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCAI
```

Session login flow

CLASSIC COOKIE-BASED LOGIN





- `authenticate()` = check the credentials. `login()` = start the session. `logout()` = end it.
- Passwords are stored hashed (PBKDF2 by default; Argon2 is an option), never as plain text.

Permissions

LEVEL	TOOL
Logged in?	`@login_required`, `LoginRequiredMixin`
Model permission	Django auto-creates add/change/delete/view for each model. Check with `user.has_perm("movies.add_movie")` or `PermissionRequiredMixin`.
Roles	**Groups**: give permissions to a group (such as "Staff"), then add users to it.
Per object ("only my rentals")	Filter the queryset by `request.user`, or use django-guardian / django-rules, or DRF `has_object_permission`.

WATCH OUT

- The safest object-level check is a scoped queryset: `Rental.objects.filter(customer=request.user)`. Then other users' data can't be found at all.
- Hiding a button is not security. Always check permissions on the server.

INTERVIEW

Why create a custom user model at the start? Show answer

AbstractUser vs AbstractBaseUser? Show answer

Authentication vs authorization? Show answer

PRACTICE: VIDLY PRO

1. Create the `users` app with a custom User **before** the first migrate.
2. Add login/logout with Django's built-in auth views.
3. Create a "Staff" group that can add movies; customers see only their own rentals.

09 The admin

A free back office. Fast to customize, but for internal staff only.

~15 min ☐ Mark as done

WHAT
An auto-generated CRUD website for your models.

WHY
Your team gets data tools on day one without building any UI.

WHEN
Internal operations and support.

HOW
Register a `ModelAdmin` in `admin.py`.

NOT
for customers.

Copy

```
class RentalInline(admin.TabularInline):
    model = Rental
    extra = 0

@admin.register(Movie)
class MovieAdmin(admin.ModelAdmin):
    list_display = ["title", "genre", "daily_rate", "number_in_stock"]
    list_filter = ["genre"]
    search_fields = ["title"]
    list_select_related = ["genre"]      # avoid N+1 in the list page
    autocomplete_fields = ["genre"]      # good for big FK tables
    readonly_fields = ["created_at"]
    inlines = [RentalInline]
    actions = ["mark_out_of_stock"]
```

```
    @admin.action(description="Set stock to 0")
    def mark_out_of_stock(self, request, queryset):
        queryset.update(number_in_stock=0)
```

WATCH OUT

- In production, change the `/admin/` URL, require strong passwords or 2FA, and give staff the smallest permissions they need.
- Big tables: use `list_select_related` and `autocomplete_fields`, or the admin will be slow.

PRACTICE: VIDLY PRO

1. Add a full `MovieAdmin` and `RentalAdmin` with filters, search, inlines and one custom action.

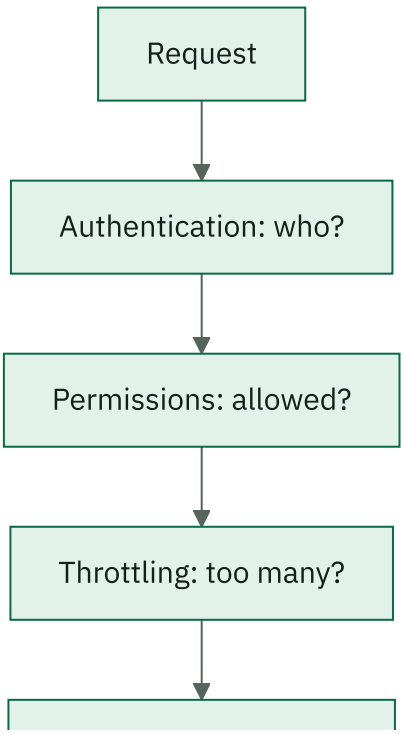
10 REST APIs with Django REST Framework

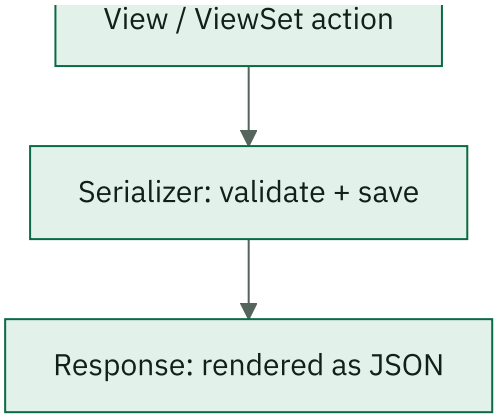
The most-used skill in Django jobs. Mobile apps and React frontends talk to these APIs.

~60 min ☐ Mark as done

How a DRF request flows

EACH STEP CAN REJECT THE REQUEST





Serializers

WHAT

Convert model ↔ JSON, and validate input. Like forms, but for APIs.

WHY

One place for the output shape and the input rules.

WHEN

`ModelSerializer` for simple CRUD. Plain `Serializer` for custom input, or separate input/output shapes.

HOW

`is_valid(raise_exception=True)`, then `save()`.

Copy

```
from rest_framework import serializers

class MovieSerializer(serializers.ModelSerializer):
    genre_name = serializers.CharField(source="genre.name", read_only=True)
    is_available = serializers.SerializerMethodField()

    class Meta:
        model = Movie
        fields = ["id", "title", "genre", "genre_name", "daily_rate", "number_
        read_only_fields = ["id"]

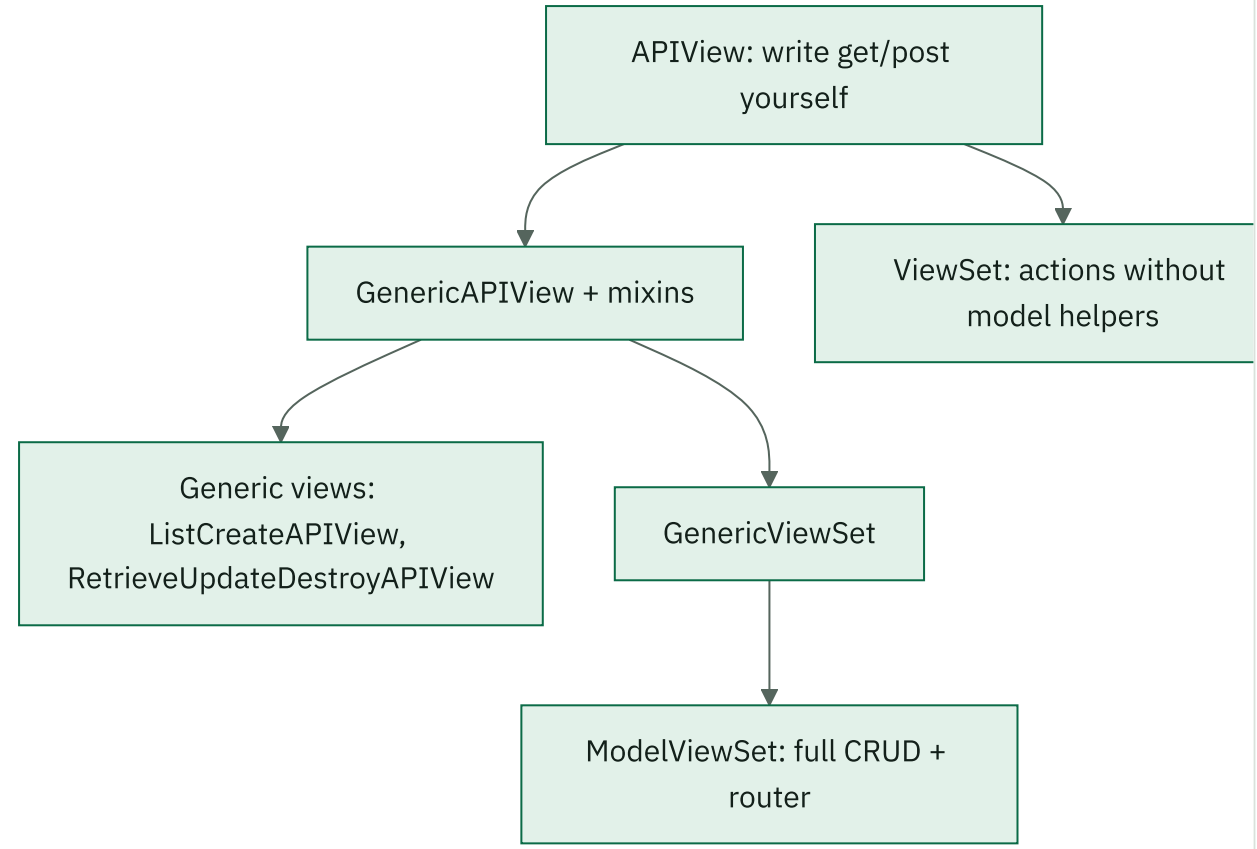
    def get_is_available(self, obj):
        return obj.number_in_stock > 0

    def validate_daily_rate(self, value):        # one field
        if value > 50:
            raise serializers.ValidationError("Too expensive.")
        return value

    def validate(self, attrs):                    # several fields
        return attrs
```

Views: pick the right level

LESS CODE GOING DOWN, LESS CONTROL



Copy

```python
from rest_framework import viewsets, permissions
from rest_framework.decorators import action
from rest_framework.response import Response
from rest_framework.routers import DefaultRouter

class MovieViewSet(viewsets.ModelViewSet):
    serializer_class = MovieSerializer
    permission_classes = [permissions.IsAuthenticatedOrReadOnly]
    filterset_fields = ["genre"]                    # django-filter
    search_fields = ["title"]
    ordering_fields = ["daily_rate", "created_at"]

    def get_queryset(self):
        return Movie.objects.select_related("genre")   # avoid N+1 here too

    def perform_create(self, serializer):
        serializer.save(created_by=self.request.user)

    @action(detail=True, methods=["post"])          # POST /movies/5/rent/
    def rent(self, request, pk=None):
        rental = rent_movie(request.user, pk)
        return Response({"rental_id": rental.id}, status=201)

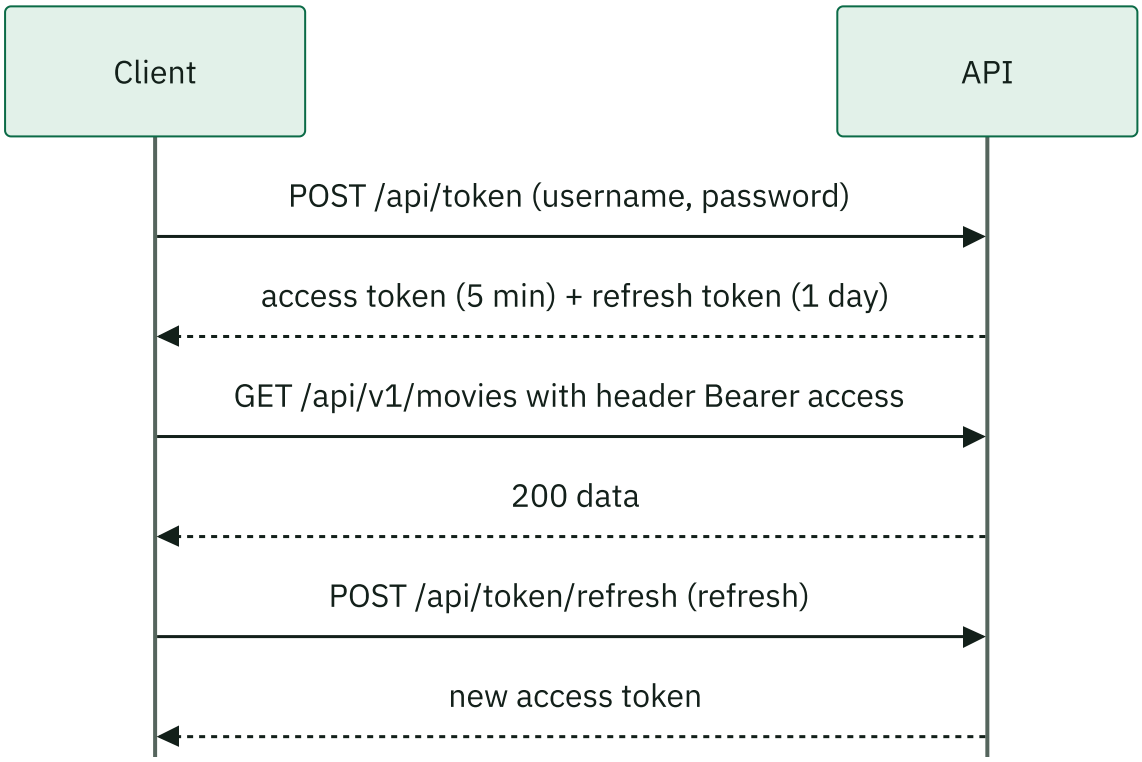
router = DefaultRouter()
```

```
router.register("movies", MovieViewSet, basename="movie")
urlpatterns = [path("api/v1/", include(router.urls))]
```

Authentication options

TYPE	BEST FOR	NOTE
Session	Same-domain web frontend	Uses cookies + CSRF.
Token (DRF built-in)	Simple internal clients	One token per user, stored in the database.
JWT (`djangorestframework-simplejwt`)	Mobile apps, SPAs, microservices	Stateless: short-lived access token + refresh token.

JWT FLOW



Settings that every real API needs

Copy

```
REST_FRAMEWORK = {
    "DEFAULT_AUTHENTICATION_CLASSES": [
        "rest_framework_simplejwt.authentication.JWTAuthentication",
    ],
    "DEFAULT_PERMISSION_CLASSES": ["rest_framework.permissions.IsAuthenticatec
    "DEFAULT_PAGINATION_CLASS": "rest_framework.pagination.PageNumberPaginatic
    "PAGE_SIZE": 20,
```

```
    "DEFAULT_FILTER_BACKENDS": [
        "django_filters.rest_framework.DjangoFilterBackend",
        "rest_framework.filters.SearchFilter",
        "rest_framework.filters.OrderingFilter",
    ],
    "DEFAULT_THROTTLE_CLASSES": ["rest_framework.throttling.UserRateThrottle"]
    "DEFAULT_THROTTLE_RATES": {"user": "1000/hour"},
    "DEFAULT_SCHEMA_CLASS": "drf_spectacular.openapi.AutoSchema",   # OpenAPI
}
```

NEED	TOOL
Pagination	`PageNumber` (simple), `LimitOffset`, `Cursor` (huge or changing lists; stable and fast)
Filtering	`django-filter`, `SearchFilter`, `OrderingFilter`
Object permission	Custom `BasePermission.has_object_permission()`
Versioning	URL path `/api/v1/` (most common) or `AcceptHeaderVersioning`
API docs	`drf-spectacular`: Swagger UI from OpenAPI 3
Error format	Custom `EXCEPTION_HANDLER`, so every error has the same JSON shape

WATCH OUT

- Serializers that touch related fields cause N+1. Always optimize `get_queryset()`.
- Set `IsAuthenticated` as the default, and open endpoints up on purpose.
- Scope data per user in `get_queryset()`. Otherwise users can read others' records by changing the ID (an IDOR bug).
- Keep business logic out of serializers. Call a service function instead (Module 15).

INTERVIEW

APIView vs GenericAPIView vs ViewSet vs ModelViewSet? Show answer

Session auth vs JWT? Show answer

How do you validate in a serializer? Show answer

How do you design an API for web, mobile and third parties? Show answer

PRACTICE: VIDLY PRO

1. Build `/api/v1/movies/` with a ModelViewSet, a router, pagination and a genre filter.
2. Add JWT login with simplejwt, and a `POST /movies/{id}/rent/` action.
3. Add `drf-spectacular` and open Swagger at `/api/docs/`.

11 Middleware and signals

Hooks around every request, and hooks around model events.

~25 min ☐ Mark as done

Middleware

WHAT

Code that runs before and after every view.

WHY

For cross-cutting jobs: auth, sessions, CSRF, logging, request IDs, timing.

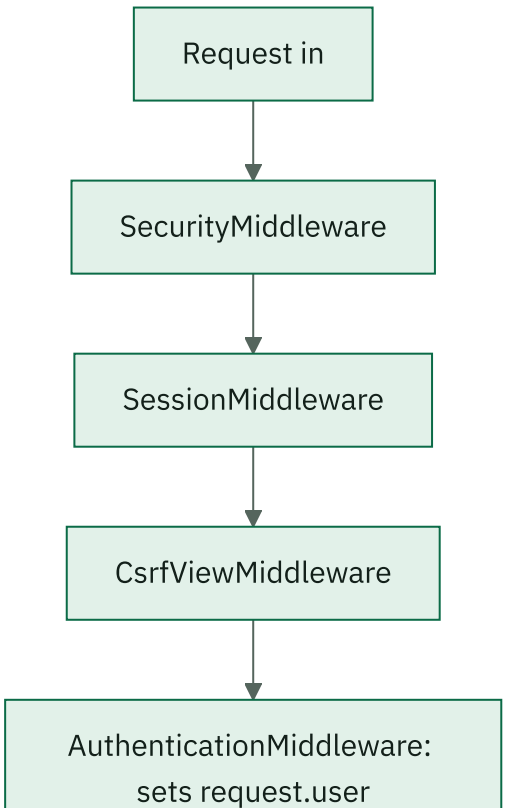
WHEN

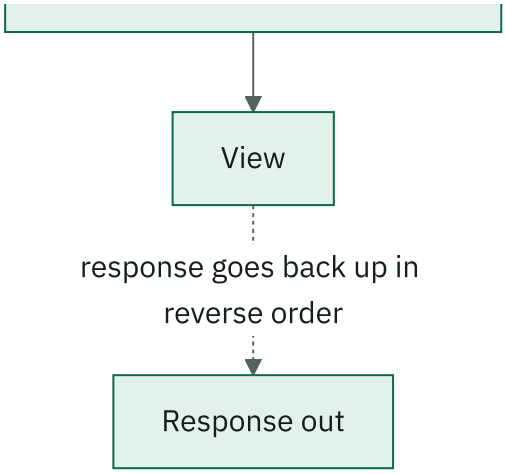
When something must happen for *all* requests. For a single view, use a decorator.

HOW

A class with `__init__(get_response)` and `__call__(request)`, added to `MIDDLEWARE`.

THE ONION: DOWN ON REQUEST, BACK UP ON RESPONSE






```python
import time, uuid

class RequestTimingMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response          # runs ONCE at startup

    def __call__(self, request):
        request.id = uuid.uuid4().hex              # BEFORE the view
        start = time.monotonic()
        response = self.get_response(request)     # calls the next layer
        ms = (time.monotonic() - start) * 1000     # AFTER the view
        response["X-Request-ID"] = request.id
        response["X-Response-Time-ms"] = f"{ms:.0f}"
        return response
```

Optional hooks: **`process_view`**, **`process_exception`**, **`process_template_response`**. Order matters. For example, **`AuthenticationMiddleware`** needs **`SessionMiddleware`** above it.

Signals

WHAT

"When X happens, also run Y." Examples: **`post_save`**, **`pre_delete`**, **`user_logged_in`**.

WHY

To react to events in code you don't own (third-party or Django apps).

WHEN TO AVOID

For your own business logic. The hidden flow is hard to debug and test. Call a service function instead.

HOW

`@receiver(post_save, sender=Rental)`, connected in **`AppConfig.ready()`**.

```python
# users/signals.py
@receiver(post_save, sender=settings.AUTH_USER_MODEL)
```

```
@receiver(post_save, sender=settings.AUTH_USER_MODEL)
def create_profile(sender, instance, created, **kwargs):
    if created:
        Profile.objects.create(user=instance)

# users/apps.py
class UsersConfig(AppConfig):
    name = "users"
    def ready(self):
        from . import signals  # noqa: connect receivers
```

WATCH OUT

- Signals do **not** fire on `queryset.update()`, `bulk_create()` or `DB_CASCADE`.
- Signals run in the same transaction and thread. A slow receiver slows every save.

INTERVIEW

What is middleware? Give real examples. Show answer

Are signals good practice? Show answer

PRACTICE: VIDLY PRO

1. Add `RequestTimingMiddleware` and log slow requests (>500 ms).
2. Create a Profile automatically for each new user with a signal.

12 Caching

Don't compute the same thing twice. Usually done with Redis.

~25 min ☐ Mark as done

WHAT

Store a result in fast memory and reuse it.

WHY

Fewer database hits and faster pages, so you need fewer servers.

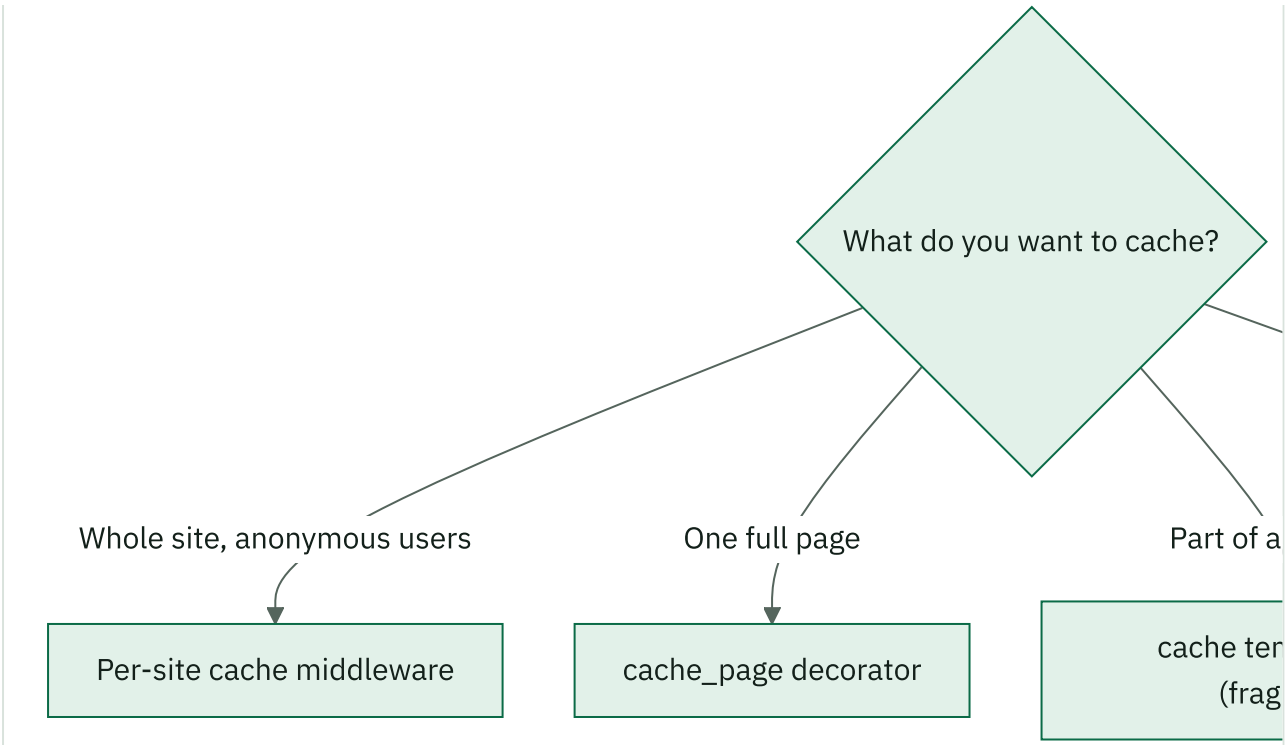
WHEN

Data that is read often, changes rarely, or is expensive to compute.

HOW

Configure `CACHES` (Redis is built in), then pick a level.

WHICH CACHE LEVEL?



Copy

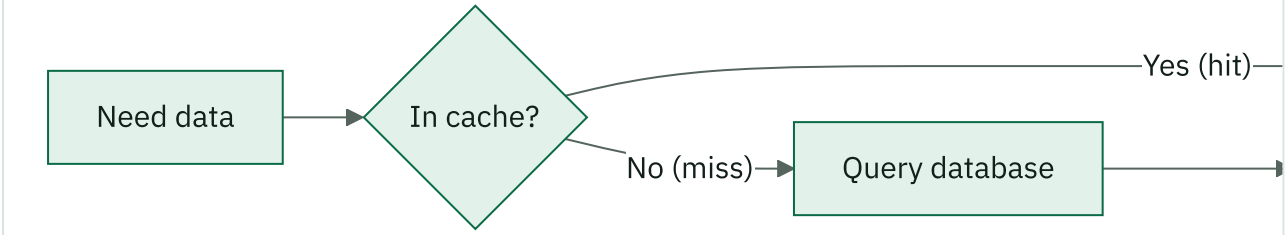
```
CACHES = {
    "default": {
        "BACKEND": "django.core.cache.backends.redis.RedisCache",
        "LOCATION": "redis://127.0.0.1:6379/1",
    }
}

from django.core.cache import cache
from django.views.decorators.cache import cache_page

@cache_page(60 * 5)                       # whole view, 5 minutes
def movie_list(request): ...

def top_movies():                          # cache-aside pattern
    data = cache.get("top_movies")
    if data is None:
        data = list(Movie.objects.order_by("-rentals_count")[:10].values("id",
        cache.set("top_movies", data, timeout=600)
    return data
```

CACHE-ASIDE (MOST COMMON)



Invalidation: the hard part

- **Timeout (TTL)**: simple. The data can be stale for up to N seconds.
- **Delete on change**: `cache.delete("top_movies")` after a save (use `on_commit`).
- **Versioned keys**: `f"movie:{id}:v{updated_at}"`. Old keys just expire.
- **Stampede**: many requests miss at once and all hit the database. Use a lock or `cache.get_or_set` with staggered TTLs.

WATCH OUT

- Never cache pages with personal data in a shared cache by mistake. Use `vary_on_cookie` or cache the data only.
- `LocMemCache` is per process. With 4 workers you get 4 different caches. Use Redis in production.

INTERVIEW

What caching levels does Django offer? Show answer

How do you keep cached data fresh? Show answer

PRACTICE: VIDLY PRO

1. Run Redis in Docker: `docker run -p 6379:6379 redis`.
2. Cache "top 10 movies" for 10 minutes; delete the key when a movie is saved.

13 Background tasks

Do slow work outside the request, so users don't wait.

~30 min ☐ Mark as done

WHAT

Send work to a queue. A separate worker process runs it.

WHY

Requests stay fast, and failed jobs can retry.

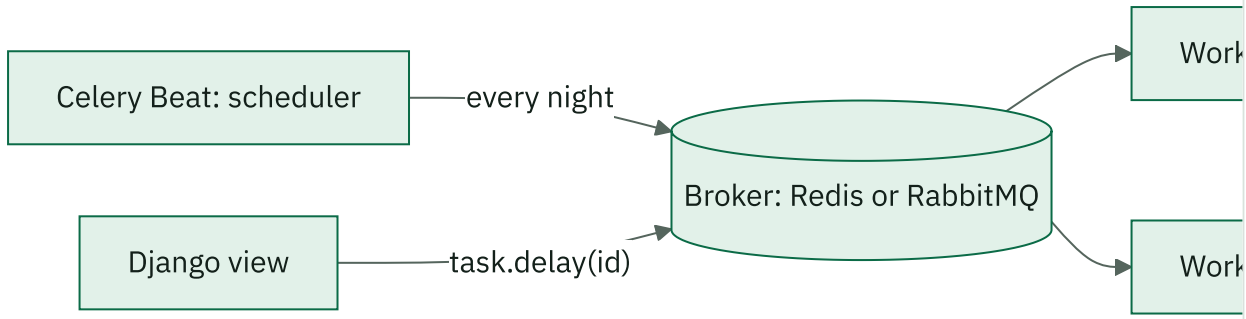
WHEN

Emails, reports, image processing, calls to slow APIs, scheduled jobs.

HOW

Celery (industry standard) or the built-in Django Tasks framework (6.0+).

CELERY ARCHITECTURE



Celery setup (short)

Copy

```
# config/celery.py
import os
from celery import Celery
os.environ.setdefault("DJANGO_SETTINGS_MODULE", "config.settings")
app = Celery("config")
app.config_from_object("django.conf:settings", namespace="CELERY")
app.autodiscover_tasks()

# config/__init__.py
from .celery import app as celery_app
__all__ = ("celery_app",)

# settings.py
CELERY_BROKER_URL = "redis://localhost:6379/0"

# rentals/tasks.py
from celery import shared_task

@shared_task(bind=True, autoretry_for=(ConnectionError,), retry_backoff=True,
def send_receipt(self, rental_id):            # pass IDs, not objects
    rental = Rental.objects.get(pk=rental_id)
    email_service.send_receipt(rental)

# in a service, after saving:
send_receipt.delay_on_commit(rental.id)       # Celery 5.4+: waits for COMMIT

# run a worker
# celery -A config worker -l INFO
```

Django Tasks (6.0+) vs Celery

DJANGO TASKS (BUILT-IN)	CELERY

API	`@task` + `.enqueue()`	`@shared_task` + `.delay()`
Worker	Not included. Needs a backend package or your own worker.	Included, mature
Retries, scheduling, routing	Basic, depends on the backend	Full: retries, beat, chains, priorities
Pick it when	New, simple projects	Enterprise work today. Most jobs ask for Celery.

WATCH OUT

- **Idempotent tasks**: a task may run twice (retry, crash). Running it again must not double-charge or double-email.
- Pass IDs, not model objects. The object may be outdated by the time the task runs.
- Keep business logic in services. The task just loads data and calls the service.
- Monitor the queue length and failures (Flower, Sentry).

INTERVIEW

How do you handle background jobs reliably? Show answer

What is idempotency and why does it matter? Show answer

PRACTICE: VIDLY PRO

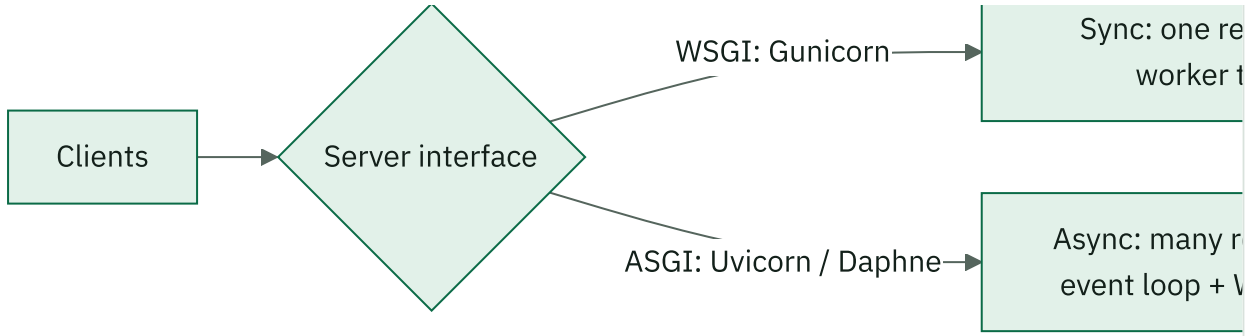
1. Set up Celery with Redis.
2. Send a rental receipt email in a task with `delay_on_commit`.
3. Add a nightly Celery Beat task that marks overdue rentals.

14 Async and real-time

ASGI, async views and WebSockets: when they help and when they don't.

~25 min ☐ Mark as done

WSGI VS ASGI



WHAT

`async def` views that can wait on I/O without blocking a worker.

WHY

Many slow external calls (APIs, LLMs) can run at the same time.

WHEN

I/O-heavy work, many outside calls, streaming, WebSockets.

NOT WHEN

Normal CRUD or CPU-heavy work. Sync is simpler and just as fast there.

Copy

```
import asyncio, httpx

async def movie_ratings(request, pk):
    movie = await Movie.objects.aget(pk=pk)              # async ORM: aget, a
    async with httpx.AsyncClient(timeout=5) as client:
        imdb, rt = await asyncio.gather(                 # two calls at the sa
            client.get(f"https://api.example.com/imdb/{movie.slug}"),
            client.get(f"https://api.example.com/rt/{movie.slug}"),
        )
    return JsonResponse({"imdb": imdb.json(), "rt": rt.json()})

# loop over a queryset asynchronously
async for m in Movie.objects.filter(genre_id=1):
    ...

# call sync code from async code
from asgiref.sync import sync_to_async
await sync_to_async(some_sync_function)()
```

Real-time with Django Channels

- **Channels** adds WebSockets: chat, live notifications, dashboards.
- **Consumer** = a "view" for a WebSocket connection. A **channel layer** (Redis) sends messages between processes.
- Simpler option for one-way updates: Server-Sent Events (SSE) or polling.

WATCH OUT

- Calling blocking code (sync ORM, `requests`) inside an async view blocks the whole event loop. Use async libraries or `sync_to_async`.
- Under ASGI, a mix of sync and async middleware causes extra thread switching.

INTERVIEW

WSGI vs ASGI? Show answer

Does async make Django faster? Show answer

PRACTICE: VIDLY PRO

1. Write an async view that calls 2 public APIs at the same time with `httpx`, and compare the timing with a sync version.

15 Enterprise architecture

How to structure a big Django codebase so a team can grow it for years.

~40 min ☐ Mark as done

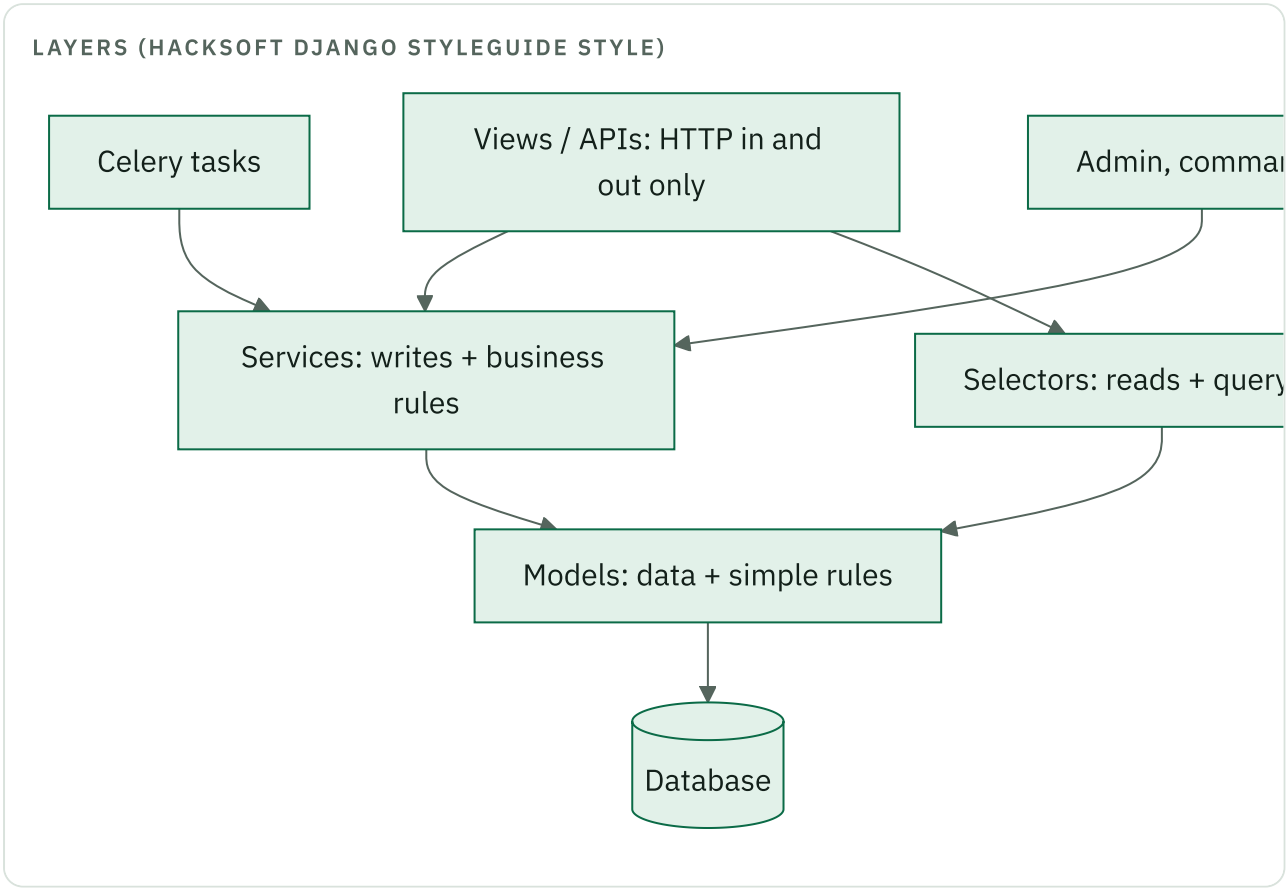
Project layout

Copy

```
vidly-pro/
├── config/
│   ├── settings/
│   │   ├── base.py          # shared
│   │   ├── local.py         # DEBUG, debug toolbar
│   │   ├── test.py          # fast password hasher, etc.
│   │   └── production.py    # security, logging, real services
│   ├── urls.py
│   ├── asgi.py / wsgi.py
│   └── celery.py
├── vidly/                   # your apps live here
│   ├── common/              # BaseModel, utils, exceptions
│   ├── users/
│   ├── movies/
│   │   ├── models.py
│   │   ├── services.py      # WRITE logic: movie_create, rent_movie
│   │   ├── selectors.py     # READ logic: movie_list, available_movies
│   │   ├── apis.py          # DRF views: thin
│   │   ├── serializers.py
│   │   ├── tasks.py
│   │   └── tests/
```

```
│   │       tests/
│   └── rentals/
├── .env.example            # committed; real .env is NOT
├── requirements/ or pyproject.toml
├── Dockerfile
└── docker-compose.yml
```

Where does business logic live?



Copy

```python
# movies/services.py
@transaction.atomic
def rental_create(*, customer: User, movie_id: int) -> Rental:
    movie = Movie.objects.select_for_update().get(pk=movie_id)
    if movie.number_in_stock < 1:
        raise ApplicationError("Movie is out of stock.")
    movie.number_in_stock -= 1
    movie.save(update_fields=["number_in_stock"])
    rental = Rental.objects.create(customer=customer, movie=movie)
    send_receipt.delay_on_commit(rental.id)
    return rental

# movies/apis.py: thin
class RentalCreateApi(APIView):
    class InputSerializer(serializers.Serializer):
        movie_id = serializers.IntegerField()

    def post(self, request):
```

```
    s = self.InputSerializer(data=request.data)
    s.is_valid(raise_exception=True)
    rental = rental_create(customer=request.user, **s.validated_data)
    return Response({"id": rental.id}, status=201)
```

APPROACH	PROS	CONS
Fat views	Fast to write	Logic can't be reused from tasks, admin or commands. Hard to test.
Fat models	The "Django way" for small apps	Models become huge, and logic across many models doesn't fit.
Services + selectors	Reusable, testable, clear	More files. Some say it's "not Django-like". Most popular for big teams.

Settings and secrets

Copy

```
# config/settings/base.py  (pip install django-environ)
import environ
env = environ.Env()
environ.Env.read_env(BASE_DIR / ".env")

SECRET_KEY = env("DJANGO_SECRET_KEY")
DEBUG = env.bool("DJANGO_DEBUG", default=False)
DATABASES = {"default": env.db("DATABASE_URL")}   # postgres://user:pass@host:
ALLOWED_HOSTS = env.list("DJANGO_ALLOWED_HOSTS", default=[])

# run with: DJANGO_SETTINGS_MODULE=config.settings.production
```

Other enterprise patterns

PATTERN	WHY / WHEN
`BaseModel` with created_at / updated_at	Every table gets timestamps for auditing and debugging.
UUID public IDs	Don't expose guessable 1, 2, 3 IDs in public URLs.
Soft delete (`deleted_at`)	Recover data and keep history. Filter it out with a custom manager.
Multiple databases + router	Send reads to a replica and writes to the primary.
Multi-tenancy	SaaS for many companies: a shared table with `tenant_id` , or a schema per tenant (django-tenants).

Structured logging	JSON logs with the request ID, so you can trace one request across services.
Custom exceptions + one error format	Clients always get `{"message": ..., "extra": ...}`.
Modular monolith first	Split into microservices only when a team or scale problem really needs it.

INTERVIEW

How would you structure a large Django project? Show answer

Where should business logic go in Django? Show answer

PRACTICE: VIDLY PRO

1. Split the settings into base/local/test/production, and move secrets to `.env`.
2. Move rental logic into `rentals/services.py`; the API and admin action both call it.

16 Testing

Tests let you change code without fear.

~35 min ☐ Mark as done

E2E / browser tests: few

API / integration tests: some

Unit tests (services, selectors, models): many

CLASS	DATABASE	USE FOR
`SimpleTestCase`	None	Pure logic, forms without the database
`TestCase`	Each test in a transaction that is rolled back (fast)	Most tests
`TransactionTestCase`	Tables truncated (slow)	Testing `on_commit`, real transactions, locks

`APITestCase` / `APIClient`	Like TestCase	DRF endpoints

pytest-django + factory_boy (the industry favorite)

Copy

```
# conftest.py / factories.py
import factory

class MovieFactory(factory.django.DjangoModelFactory):
    class Meta:
        model = Movie
    title = factory.Faker("sentence", nb_words=3)
    genre = factory.SubFactory(GenreFactory)
    daily_rate = 3
    number_in_stock = 1

# tests/test_services.py
import pytest

@pytest.mark.django_db
def test_rental_create_reduces_stock(user):
    movie = MovieFactory(number_in_stock=1)
    rental_create(customer=user, movie_id=movie.id)
    movie.refresh_from_db()
    assert movie.number_in_stock == 0

@pytest.mark.django_db
def test_rental_fails_when_out_of_stock(user):
    movie = MovieFactory(number_in_stock=0)
    with pytest.raises(ApplicationError):
        rental_create(customer=user, movie_id=movie.id)

# tests/test_apis.py
@pytest.mark.django_db
def test_movie_list_queries(api_client, django_assert_num_queries):
    MovieFactory.create_batch(5)
    with django_assert_num_queries(2):          # count + page: no N+1
        r = api_client.get("/api/v1/movies/")
    assert r.status_code == 200
```

- **Mock** external things (payment, email, HTTP) with `unittest.mock.patch` or `responses`.
- **Coverage**: `pytest --cov`. Aim for high coverage on services, not 100% everywhere.
- **CI**: run the tests and `makemigrations --check` on every pull request.

INTERVIEW

TestCase vs TransactionTestCase? Show answer

What is your testing approach? Show answer

PRACTICE: VIDLY PRO

1. Install pytest-django + factory_boy.
2. Test `rental_create` (success, out of stock) and the movies API (auth, filter, query count).
3. Run them in GitHub Actions.

17 Security

Django protects you by default. Know what it does, and what is still your job.

~25 min ☐ Mark as done

ATTACK	DJANGO'S PROTECTION	YOUR JOB
SQL injection	The ORM uses query parameters	Never put user input into raw SQL with f-strings
XSS	Templates auto-escape	Avoid `|safe` and `mark_safe` on user input; add CSP
CSRF	`CsrfViewMiddleware` + token	Use `{% csrf_token %}`; don't use `@csrf_exempt` without a reason
Clickjacking	`XFrameOptionsMiddleware`	Keep it on
Host header attacks	`ALLOWED_HOSTS`	Set it to your real domains
Weak passwords	Password validators + hashing	Rate-limit logins (django-axes), offer 2FA
IDOR (seeing others' data)	Nothing automatic	Scope querysets to `request.user`
Leaked secrets	Nothing automatic	Environment variables, `.gitignore`, rotate keys with `SECRET_KEY_FALLBACKS`

Production security settings

Copy

```
DEBUG = False
ALLOWED_HOSTS = ["vidly.com"]
SECURE_SSL_REDIRECT = True
SESSION_COOKIE_SECURE = True
CSRF_COOKIE_SECURE = True
SECURE_HSTS_SECONDS = 31536000            # start small, then increase
SECURE_HSTS_INCLUDE_SUBDOMAINS = True
SECURE_PROXY_SSL_HEADER = ("HTTP_X_FORWARDED_PROTO", "https")   # behind Ngin

# Django 6.0+: Content Security Policy
from django.utils.csp import CSP
MIDDLEWARE += ["django.middleware.csp.ContentSecurityPolicyMiddleware"]
SECURE_CSP = {"default-src": [CSP.SELF], "script-src": [CSP.SELF, CSP.NONCE]}

# Then run:
# python manage.py check --deploy
```

WATCH OUT

- `DEBUG = True` in production shows your settings and code to everyone on error pages.
- Scan dependencies regularly: `pip-audit`, Dependabot.

INTERVIEW

How does Django prevent SQL injection and XSS? Show answer

What do you check before going live? Show answer

PRACTICE: VIDLY PRO

1. Run `check --deploy` with production settings and fix every warning.
2. Add CSP and login rate limiting.

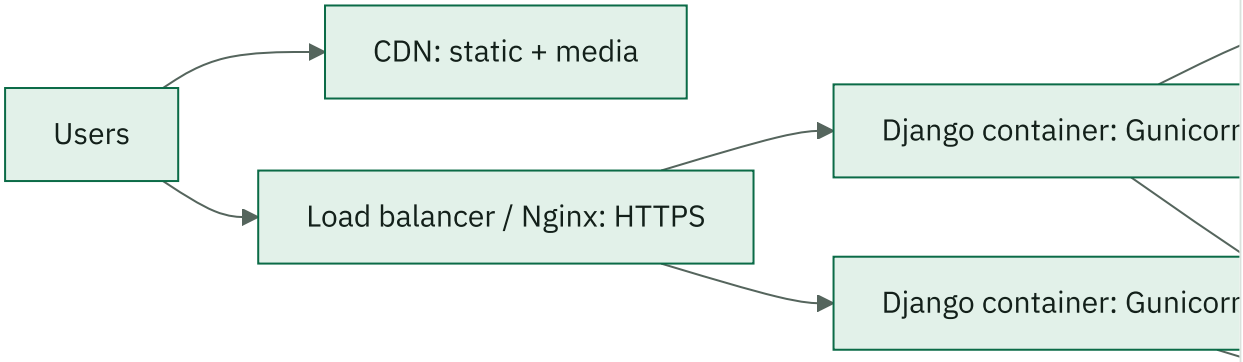
18 Deployment and operations

From laptop to a production system that scales and can be watched.

~40 min ☐ Mark as done

Production architecture

A TYPICAL ENTERPRISE SETUP



Docker

Copy

```
# Dockerfile
FROM python:3.12-slim
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
RUN python manage.py collectstatic --noinput
RUN useradd -m app && chown -R app /app
USER app                                    # never run as root
CMD ["gunicorn", "config.wsgi:application", "--bind", "0.0.0.0:8000", "--worke
# async app: gunicorn config.asgi:application -k uvicorn.workers.UvicornWorke
```

With `docker-compose.yml`, run **web**, **db** (Postgres), **redis** and **worker** (Celery) together locally. A common Gunicorn starting point is `workers = 2 × CPU cores + 1`.

Static and media files

STATIC

Your CSS/JS/images. `collectstatic` copies them to `STATIC_ROOT`. Serve them with WhiteNoise, Nginx or a CDN.

MEDIA

Files users upload. Store them in S3 or similar (django-storages), never on the container disk.

WHY

Containers are temporary. Files saved inside them are lost on redeploy.

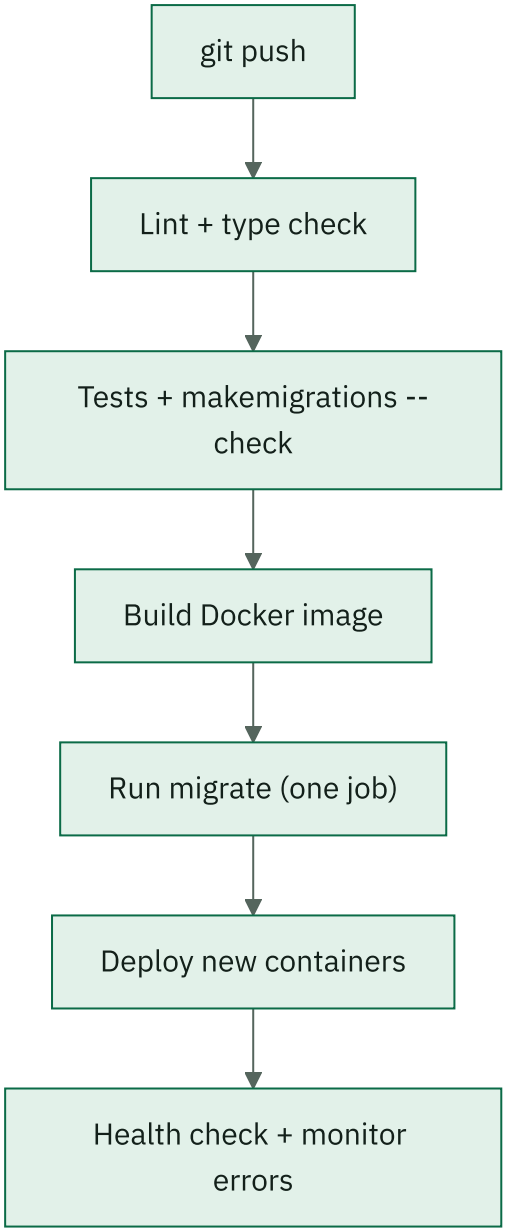
RULE

Django should not serve files in production. Let Nginx, a CDN or WhiteNoise do it.

CI/CD pipeline

CI/CD pipeline

EVERY PUSH TO MAIN



How to scale, step by step

1. **Measure** with an APM tool and logs. Find the real bottleneck.
2. **Fix queries**: N+1, indexes, `only()`.
3. **Cache** hot data in Redis; put a CDN in front of static files.
4. **Move slow work** to Celery.
5. **Add app servers** behind a load balancer. Django is stateless when sessions and files are kept outside the server.
6. **Scale the database**: connection pooling (PgBouncer, or the Postgres pool option in Django 5.1+), read replicas, then partitioning.

INTERVIEW

How do you deploy Django to production? Show answer

How do you scale a Django app? Show answer

Static vs media files? Show answer

PRACTICE: VIDLY PRO

1. Write a Dockerfile and a docker-compose file (web, db, redis, worker).
2. Add a GitHub Actions workflow: tests, then build.
3. Deploy to Render, Railway or Fly.io, and connect Sentry.

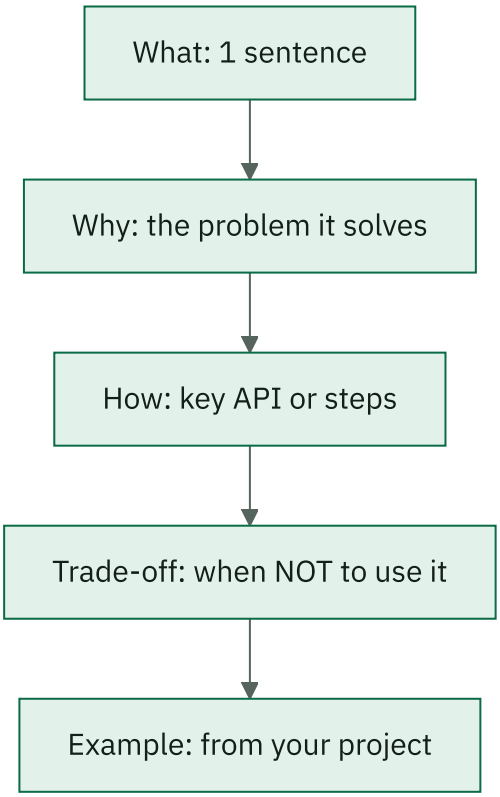
19 Interview prep

Rapid-fire questions, a simple answer formula and a cheat sheet for revision.

~45 min, then repeat ☐ Mark as done

The answer formula

USE THIS FOR EVERY QUESTION



RAPID FIRE: CORE

null=True vs blank=True? Show answer

What are Django's main built-in protections? Show answer

What is a context processor? Show answer

What does AppConfig.ready() do? Show answer

What is a manager vs a QuerySet? Show answer

Abstract vs proxy vs multi-table inheritance? Show answer

How do you handle a ForeignKey delete? Show answer

RAPID FIRE: ORM AND PERFORMANCE

What is the N+1 problem and how do you fix it? Show answer

How do you update a counter safely? Show answer

How do you work with millions of rows? Show answer

What does select_for_update do? Show answer